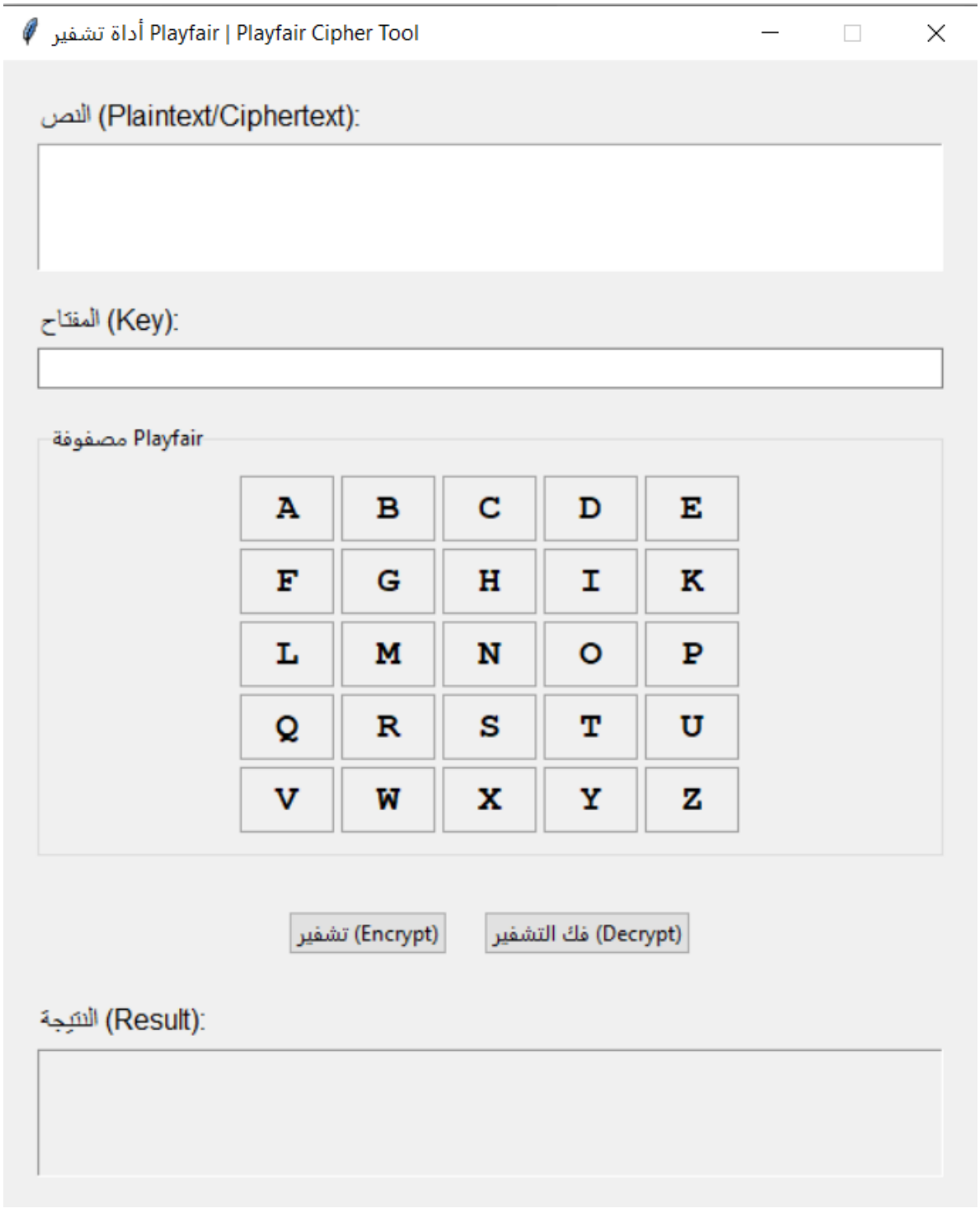
Playfair Alogritm
IT4 Ibb UN
ENG: Nader AL-hoshaee



```python
import tkinter as tk
from tkinter import ttk, messagebox, font

# ================================================================
#  الجزء الأول: الدوال الأساسية لـ Playfair (بعد إعادة الهيكلة)
#  (Part 1: Core Playfair functions - refactored for clarity)
# ================================================================

def create_playfair_matrix(key:str):
    """
    تنشئ مصفوفة Playfair (5x5) بناءً على المفتاح.
    تم إخراج هذه الدالة لتجنب تكرارها في كل من التشفير وفك التشفير.
    """
    alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ"  # استبدال "J" بـ "I"
    key = key.upper().replace("J", "I")
    matrix_chars = []
    used = set()

    # إضافة الأحرف الفريدة من المفتاح إلى المصفوفة
    for char in key:
        if char.isalpha() and char not in used:
            matrix_chars.append(char)
            used.add(char)

    # إضافة باقي الأحرف من الأبجدية
    for char in alphabet:
        if char not in used:
            matrix_chars.append(char)

    # تحويل القائمة إلى مصفوفة 5x5
    return [matrix_chars[i : i + 5] for i in range(0, 25, 5)]

def find_position(matrix, char):
    """
    تجد موقع (صف، عمود) لحرف معين في المصفوفة.
    """
    for row in range(5):
        for col in range(5):
            if matrix[row][col] == char:
                return row, col
    return None

def playfair_encrypt(text, key):
```

```
"""
تشفير النص باستخدام شيفرة Playfair.
"""
matrix = create_playfair_matrix(key)

# تحضير النص: إزالة غير الحروف، استبدال J بـ I، وتحويله لحروف كبيرة
text = text.upper().replace("J", "I")
cleaned_text = "".join(filter(str.isalpha, text))

# إضافة "X" بين الحروف المكررة في زوج واحد
i = 0
temp_text = ""
while i < len(cleaned_text):
    char1 = cleaned_text[i]
    if i + 1 == len(cleaned_text):
        temp_text += char1 + 'X'
        break
    char2 = cleaned_text[i+1]
    if char1 == char2:
        temp_text += char1 + 'X'
        i += 1
    else:
        temp_text += char1 + char2
        i += 2
cleaned_text = temp_text

# تشفير النص
ciphertext = ""
for i in range(0, len(cleaned_text), 2):
    a = cleaned_text[i]
    b = cleaned_text[i + 1]

    row_a, col_a = find_position(matrix, a)
    row_b, col_b = find_position(matrix, b)

    if row_a == row_b:  # نفس الصف
        ciphertext += matrix[row_a][(col_a + 1) % 5]
        ciphertext += matrix[row_b][(col_b + 1) % 5]
    elif col_a == col_b:  # نفس العمود
        ciphertext += matrix[(row_a + 1) % 5][col_a]
        ciphertext += matrix[(row_b + 1) % 5][col_b]
    else:  # مستطيل
        ciphertext += matrix[row_b][col_a]
        ciphertext += matrix[row_a][col_b]
```

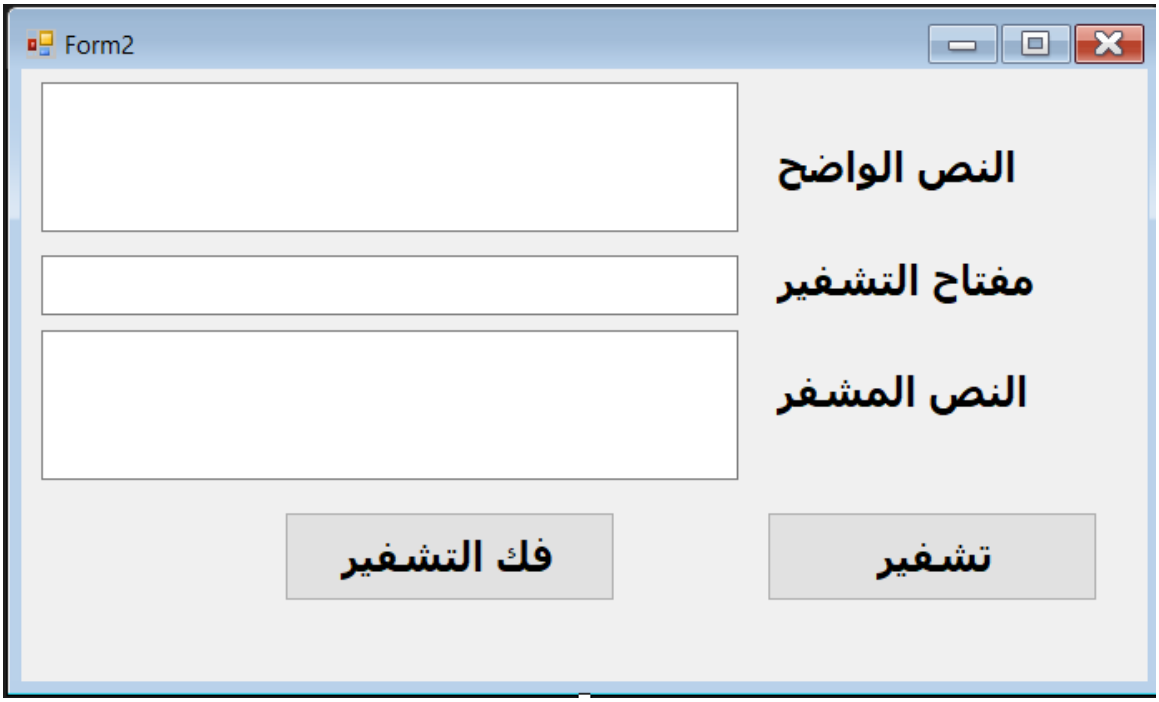
```
    # إرجاع النص المشفر مقسماً إلى أزواج
    return " ".join([ciphertext[j:j+2] for j in range(0, len(ciphertext),
2)])

def playfair_decrypt(ciphertext, key):
    """
    فك تشفير النص باستخدام شيفرة Playfair.
    """
    matrix = create_playfair_matrix(key)
    ciphertext = "".join(ciphertext.upper().split())
    plaintext = ""

    for i in range(0, len(ciphertext), 2):
        a = ciphertext[i]
        b = ciphertext[i + 1]

        row_a, col_a = find_position(matrix, a)
        row_b, col_b = find_position(matrix, b)

        if row_a == row_b:  # نفس الصف
            plaintext += matrix[row_a][(col_a - 1) % 5]
            plaintext += matrix[row_b][(col_b - 1) % 5]
        elif col_a == col_b:  # نفس العمود
            plaintext += matrix[(row_a - 1) % 5][col_a]
            plaintext += matrix[(row_b - 1) % 5][col_b]
        else:  # مستطيل
            plaintext += matrix[row_b][col_a]
            plaintext += matrix[row_a][col_b]

    return plaintext

# ================================================================
#  الجزء الثاني: بناء الواجهة الرسومية وربطها بالمنطق
#  (Part 2: Building the GUI and connecting the logic)
# ================================================================

class PlayfairApp:
    def __init__(self, root):
        self.root = root
        self.root.title("أداة تشفير Playfair | Playfair Cipher Tool")
        self.root.geometry("550x650")
        self.root.resizable(False, False)

        # إعداد الإطار الرئيسي
        main_frame = ttk.Frame(self.root, padding="20")
```

```
        main_frame.pack(fill="both", expand=True)

        # --- عناصر الواجهة ---

        # 1. إدخال النص
        ttk.Label(main_frame, text="النص (Plaintext/Ciphertext):",
font=("Arial", 12)).pack(anchor="w", pady=(0, 5))
        self.input_text = tk.Text(main_frame, height=4, font=("Arial",
11), wrap="word")
        self.input_text.pack(fill="x", expand=True)

        # 2. إدخال المفتاح
        ttk.Label(main_frame, text="المفتاح (Key):", font
        =("Arial", 12)).pack(anchor="w", pady=(15, 5))
        self.key_entry = ttk.Entry(main_frame, font=("Arial", 11))
        self.key_entry.pack(fill="x")
        # ربط حدث تغيير المفتاح بتحديث المصفوفة
        self.key_entry.bind("<KeyRelease>", self.update_matrix_display)

        # 3. عرض مصفوفة Playfair
        self.matrix_frame = ttk.LabelFrame(main_frame, text="مصفوفة
Playfair", padding="10")
        self.matrix_frame.pack(pady=20, fill="x")

        self.matrix_labels = []
        matrix_font = font.Font(family="Courier New", size=16,
weight="bold")
        grid_frame = ttk.Frame(self.matrix_frame)
        grid_frame.pack()
        for i in range(5):
            row_labels = []
            for j in range(5):
                label = ttk.Label(grid_frame, text="", font=matrix_font,
width=3, anchor="center", relief="solid", padding=5)
                label.grid(row=i, column=j, padx=2, pady=2)
                row_labels.append(label)
            self.matrix_labels.append(row_labels)

        # 4. الأزرار
        button_frame = ttk.Frame(main_frame)
        button_frame.pack(pady=10)
        ttk.Button(button_frame, text="تشفير (Encrypt)",
command=self.perform_encrypt).pack(side="left", padx=10)
        ttk.Button(button_frame, text="فك التشفير (Decrypt)",
command=self.perform_decrypt).pack(side="right", padx=10)
```

```
        # 5. عرض النتيجة
        ttk.Label(main_frame, text="النتيجة (Result):", font=("Arial",
12)).pack(anchor="w", pady=(15, 5))
        self.output_text = tk.Text(main_frame, height=4, font=("Arial",
11), wrap="word", state=tk.DISABLED, bg="#f0f0f0")
        self.output_text.pack(fill="x", expand=True)

        # تحديث أولي للمصفوفة عند بدء التشغيل
        self.update_matrix_display()

    def update_matrix_display(self, event=None):
        """تحديث عرض المصفوفة 5x5 بناءً على المفتاح المدخل."""
        key = self.key_entry.get()
        matrix = create_playfair_matrix(key)
        for i in range(5):
            for j in range(5):
                self.matrix_labels[i][j].config(text=matrix[i][j])

    def perform_encrypt(self):
        text = self.input_text.get("1.0", tk.END).strip()
        key = self.key_entry.get().strip()
        if not text or not key:
            messagebox.showwarning("إدخال ناقص", "الرجاء إدخال النص والمفتاح.")
            return

        try:
            result = playfair_encrypt(text, key)
            self.output_text.config(state=tk.NORMAL)
            self.output_text.delete("1.0", tk.END)
            self.output_text.insert("1.0", result)
            self.output_text.config(state=tk.DISABLED)
        except Exception as e:
            messagebox.showerror("خطأ", f"حدث خطأ أثناء التشفير: {e}")

    def perform_decrypt(self):
        text = self.input_text.get("1.0", tk.END).strip()
        key = self.key_entry.get().strip()
        if not text or not key:
            messagebox.showwarning("إدخال ناقص", "الرجاء إدخال النص والمفتاح.")
            return

        try:
            result = playfair_decrypt(text, key)
            self.output_text.config(state=tk.NORMAL)
```

```
            self.output_text.delete("1.0", tk.END)
            self.output_text.insert("1.0", result)
            self.output_text.config(state=tk.DISABLED)
        except Exception as e:
            messagebox.showerror("خطأ", f"حدث خطأ أثناء فك التشفير: {e}")


if __name__ == "__main__":
    root = tk.Tk()
    app = PlayfairApp(root)
    root.mainloop()
```

خوارزمية بلايفر سي شارب



```
public partial class Form2 : Form
    {
        public Form2()
        {
            InitializeComponent();
        }

        private void btnEncrypt_Click(object sender, EventArgs e)
        {
            string plaintext = txtInput.Text.ToUpper().Replace("J", "I").Replace("
", "X");
            string keyword = txtKey.Text.ToUpper().Replace("J", "I").Replace(" ",
"");

            string ciphertext = EncryptPlayfair(plaintext, keyword,true);

            // The original code removes the filler 'X' for display, which might
be misleading
            // but we will keep it as it is in the image.
            txtOutput.Text = ciphertext;
        }

        private void btnDecrypt_Click(object sender, EventArgs e)
        {
            // Assuming txtInput is used for CipherText in this case.
            string ciphertext = txtInput.Text.ToUpper().Replace(" ", "");
            string keyword = txtKey.Text.ToUpper().Replace("J", "I").Replace(" ",
"");

            string decryptedText = EncryptPlayfair(ciphertext, keyword,false);
            txtOutput.Text = decryptedText;
        }

        // --- Main Encryption/Decryption Wrappers ---

        private string EncryptPlayfair(string plaintext, string keyword,bool b)
        {
            char[,] matrix = GenerateMatrix(keyword);
            return ProcessText(plaintext, matrix, b); // true for encryption
        }

        private string DecryptPlayfair(string ciphertext, string keyword)
        {
            char[,] matrix = GenerateMatrix(keyword);
            return ProcessText(ciphertext, matrix, false); // false for decryption
        }

        // --- Core Logic ---

        private string ProcessText(string text, char[,] matrix, bool encrypt)
        {
            string result = "";

            // 1. Prepare the text by pairing letters and handling duplicates/odd
length
            for (int i = 0; i < text.Length; i += 2)
```

```
            {
                if (i + 1 >= text.Length || text[i]==text[i+1])
                {
                    text = text.Insert(i + 1, "X");
                }

                char a = text[i];
                char b = text[i + 1];

                (int row1, int col1) = FindPosition(matrix, a);
                (int row2, int col2) = FindPosition(matrix, b);

                // 2. Apply Playfair rules
                if (row1 == row2) // Same row
                {
                    int direction = encrypt ? 1 : 4; // 4 is equivalent to -1 in
modulo 5
                    result += matrix[row1, (col1 + direction) % 5];
                    result += matrix[row2, (col2 + direction) % 5];
                }
                else if (col1 == col2) // Same column
                {
                    int direction = encrypt ? 1 : 4;
                    result += matrix[(row1 + direction) % 5, col1];
                    result += matrix[(row2 + direction) % 5, col2];
                }
                else // Rectangle
                {
                    result += matrix[row2, col1];
                    result += matrix[row1, col2];
                }
            }
            return result;
        }

        // --- Helper Functions ---

        private char[,] GenerateMatrix(string keyword)
        {
            char[,] matrix = new char[5, 5];
            string usedChars = "";

            // Add unique characters from the keyword
            foreach (char c in keyword)
            {
                if (!usedChars.Contains(c) && c != 'J')
                {
                    usedChars += c;
                }
            }

            // Add remaining characters of the alphabet
            string alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ";
            foreach (char c in alphabet)
            {
                if (!usedChars.Contains(c))
                {
                    usedChars += c;
```

```
            }
        }

        // Populate the 5x5 matrix
        int index = 0;
        for (int row = 0; row < 5; row++)
        {
            for (int col = 0; col < 5; col++)
            {
                matrix[row, col] = usedChars[index++];
            }
        }
        return matrix;
    }

    private (int, int) FindPosition(char[,] matrix, char c)
    {
        for (int row = 0; row < 5; row++)
        {
            for (int col = 0; col < 5; col++)
            {
                if (matrix[row, col] == c)
                {
                    return (row, col);
                }
            }
        }
        return (-1, -1);
    }


}
```